

Construção de um Compilador para Fortran 77 Standard

Ana Inês Leite, a116261

José Pereira, a89596

Ricardo Veloso, a69239

Universidade do Minho || Processamento de Linguagens

1 Introdução

O presente projeto teve como objetivo desenvolver, em Python, um compilador para a linguagem Fortran 77. A solução implementa as principais fases de compilação, nomeadamente análise léxica, análise sintática, análise semântica e geração direta de código para uma máquina virtual.

O compilador recebe um programa escrito numa sintaxe Fortran 77 e, caso não sejam detetados erros, produz uma sequência de instruções para a máquina virtual. A implementação está organizada em módulos: o *lexer* identifica os tokens, o *parser* constrói a árvore sintática abstrata, o analisador semântico valida a coerência do programa e o gerador de código produz as instruções finais para a máquina virtual.

O subconjunto implementado inclui declarações dos tipos `INTEGER`, `REAL` e `LOGICAL`, variáveis escalares e arrays unidimensionais, atribuições, expressões aritméticas, relacionais e lógicas, condicionais `IF-THEN` e `IF-THEN-ELSE`, ciclos `DO` com labels, `GOTO`, `CONTINUE` e operações básicas de entrada e saída com `READ`, `PRINT` e `WRITE`.

2 Análise Léxica

A análise léxica foi implementada com recurso à biblioteca `ply.lex` e tem como função transformar o código fonte numa sequência de tokens, posteriormente utilizada pelo analisador sintático. O *lexer* reconhece palavras reservadas, identificadores, literais, operadores e símbolos especiais.

As palavras reservadas reconhecidas são `PROGRAM`, `END`, `INTEGER`, `REAL`, `LOGICAL`, `IF`, `THEN`, `ELSE`, `ENDIF`, `DO`, `CONTINUE`, `GOTO`, `READ`, `WRITE` e `PRINT`. Para além destas, são definidos tokens para operadores aritméticos, relacionais e lógicos, atribuição, parênteses e vírgulas.

Os identificadores são reconhecidos por uma regra que exige que o primeiro caracter seja uma letra, podendo depois conter letras, dígitos ou underscores. O valor textual é convertido para maiúsculas, uniformizando a representação interna. Esta regra verifica se a sequência corresponde a uma palavra reservada, caso contrário, o token é classificado como `ID`.

Os literais inteiros são convertidos para `int`, os reais para `float` e os literais lógicos `.TRUE.` e `.FALSE.` para `True` e `False`, respetivamente. As strings são delimitadas por plicas, sendo removidos os delimitadores exteriores. O lexer também trata plicas duplicadas no interior de strings, convertendo-as numa única plica.

O lexer ignora espaços, tabulações e o caracter `\r`. As quebras de linha atualizam o número da linha corrente e os comentários iniciados por `!` são descartados. Quando surge um caracter ilegal, é registada uma mensagem de erro com a linha e o caracter encontrado. Depois disso, o lexer avança uma posição na entrada. No fluxo principal do compilador, a existência de erros léxicos impede a continuação da compilação.

3 Análise Sintática

A análise sintática foi implementada com recurso à biblioteca `ply.yacc`. O parser define a precedência dos operadores para evitar ambiguidades na interpretação de expressões. A ordem estabelecida, da menor para a maior precedência, é: `.OR.`, `.AND.`, `.NOT.`, operadores relacionais, adição e subtração, multiplicação e divisão, e menos unário.

A gramática reconhece programas com a estrutura `PROGRAM ID body END`. O corpo do programa é constituído por declarações, seguidas de instruções. As declarações suportam os tipos `INTEGER`, `REAL` e `LOGICAL`, podendo incluir variáveis escalares ou arrays unidimensionais declarados na forma `ID (INT_LITERAL)`.

As instruções incluem atribuições, estruturas condicionais, ciclos `DO`, saltos `GOTO`, instruções de I/O, labels e `CONTINUE`. As atribuições são compostas por uma variável, escalar ou indexada, seguida de `=` e de uma expressão.

As estruturas condicionais podem assumir as formas `IF (expressao) THEN ... ENDIF` ou `IF (expressao) THEN ... ELSE ... ENDIF`. Os ciclos são reconhecidos na forma `DO label variavel = expressao, expressao`. A gramática não prevê um terceiro parâmetro para o passo do ciclo, pelo que na geração de código, o incremento é sempre unitário.

As instruções `READ`, `PRINT` e `WRITE` são aceites com ou sem especificador `*`. Esse formato é reconhecido sintaticamente, mas não é guardado como informação relevante na árvore sintática abstrata. A AST conserva apenas as listas de variáveis, no caso do `READ`, ou de expressões, no caso de `PRINT` e `WRITE`.

A árvore sintática abstrata é representada através de tuplos, listas e dicionários, uma opção simples e adequada à dimensão do compilador desenvolvido. Esta representação permite identificar facilmente o tipo de cada nó e os respetivos componentes, facilitando o percurso da árvore nas fases seguintes, nomeadamente na análise semântica e na geração de código. O nó principal tem a forma `("program", nome, corpo)`, sendo o corpo um dicionário com as chaves `declarations` e `statements`. As declarações usam a forma `("decl", tipo, lista)`, os arrays declarados são representados por `("array_decl", nome, tamanho)`, as variáveis por `("var", nome)` e os acessos a arrays por `("array_ref", nome, indice)`.

As restantes instruções seguem igualmente uma representação uniforme: atribuições como `("assign", variavel, expressao)`, condicionais como `("if-then", ...)` ou `("if-then-else", ...)`, ciclos como `("do-loop", label, variavel, inicio, fim)`, saltos como `("goto", label)`, labels como `("label", numero, instrucao)` e `CONTINUE` como `("continue", )`. As expressões binárias são representadas por `("binop", operador, esquerda, direita)`, as unárias por `("unop", operador, expressao)`, os literais por `("leaf", valor)` e as strings por `("string", valor)`.

O parser é criado através de `yacc.yacc()`. Quando ocorre um erro sintático, é apresentada uma mensagem com o token inesperado e a respetiva linha. Por outro lado, se o erro resultar do fim inesperado do ficheiro, é apresentada uma mensagem específica. No main, se a AST não for gerada, a compilação é interrompida.

4 Análise Semântica

A análise semântica é realizada pela classe `SemanticAnalyzer`, que verifica a coerência do programa depois da construção da AST. Para isso, possui uma tabela de símbolos, uma lista de erros e conjuntos auxiliares usados no controlo de labels, incluindo labels definidas, labels usadas, labels associadas a ciclos DO e labels ligadas a instruções CONTINUE.

Numa primeira fase, o analisador confirma se a AST corresponde a um programa válido e percorre as declarações. Para cada identificador declarado, são registados o tipo, a natureza da variável, que pode ser escalar ou array, a dimensão, quando aplicável, e um campo de inicialização. Nesta etapa, são também detetadas variáveis redeclaradas e arrays com dimensão inválida.

Durante a análise das instruções, são verificadas as atribuições, as estruturas condicionais, os ciclos, as labels e as instruções de entrada e saída. Nas atribuições, o analisador confirma se o lado esquerdo corresponde a uma variável válida e se o tipo da expressão é compatível com o tipo da variável. A compatibilidade definida aceita tipos iguais e permite atribuir valores INTEGER a variáveis REAL.

Este analisador valida, ainda, o uso de variáveis e arrays. São detetadas variáveis não declaradas, arrays usados sem índice, variáveis escalares usadas como arrays e índices de arrays cujo tipo não seja INTEGER. Nas instruções IF, a condição tem obrigatoriamente de ser do tipo LOGICAL.

Nos ciclos `DO`, a variável de controlo tem de estar declarada, ser escalar e ter tipo `INTEGER`. Os valores inicial e final do ciclo têm de ser numéricos, sendo aceites expressões dos tipos `INTEGER` ou `REAL`. No final da análise, o compilador verifica se todas as labels usadas por `GOTO` ou `DO` foram definidas. Além disso, quando uma label é usada para terminar um ciclo `DO`, essa label tem de corresponder a uma instrução `CONTINUE`.

As expressões são analisadas de forma recursiva. Os operadores aritméticos exigem operandos numéricos, os operadores relacionais exigem operandos comparáveis e produzem valores lógicos, e os operadores lógicos exigem operandos `LOGICAL`. O menos unário exige um operando numérico, enquanto `.NOT.` exige um operando lógico. Os erros semânticos são acumulados e, caso existam, impedem a geração de código.

5 Tradução de Código

A tradução de código é realizada pela classe `CodeGenerator`, que recebe a AST e a tabela de símbolos e gera diretamente instruções da máquina virtual. Antes de traduzir o corpo do programa, é construído o esquema de memória global. As variáveis escalares ocupam uma posição de memória e os arrays ocupam o número de posições indicado na declaração. Quando existe memória global a reservar, é emitida a instrução `PUSHN`.

Nas atribuições a variáveis escalares, é gerado primeiro o código da expressão e depois a instrução `STOREG`, que guarda o valor no endereço global da variável. Nas atribuições a arrays, o gerador calcula o endereço da posição pretendida, gera o valor a armazenar e emite `STOREN`. O acesso a arrays usa o endereço base e ajusta o índice com `-1`, refletindo a indexação iniciada em 1. A leitura de uma posição de array é feita com `LOADN`.

As estruturas `IF` são traduzidas com labels internas geradas automaticamente e saltos condicionais `JZ`. No caso de `IF-THEN-ELSE`, são criadas labels para o bloco `ELSE` e para o fim da estrutura. Os ciclos `DO` inicializam a variável de controlo, comparam-na com

o limite final através de `INFEQ`, executam o corpo enquanto a condição for verdadeira e incrementam a variável em uma unidade no final de cada iteração.

As instruções `GOTO` são traduzidas para `JUMP`, e as labels do programa são emitidas na forma `L<label>:`. As instruções `READ` geram uma leitura seguida de conversão, usando `ATOF` para destinos do tipo `REAL` e `atoi` nos restantes casos. O valor lido é guardado com `STOREG`, se o destino for uma variável escalar, ou com `STOREN`, se for uma posição de array.

As instruções `PRINT` e `WRITE` usam o mesmo processo de geração de saída. Cada expressão é avaliada e depois escrita com `WRITES`, `WRITEF` ou `WRITEI`, consoante se trate de uma string, de um valor real ou de outro tipo. As expressões são traduzidas segundo um modelo de pilha, recorrendo a instruções como `PUSHI`, `PUSHF`, `PUSHS`, `PUSHG`, `ADD`, `SUB`, `MUL`, `DIV`, `EQUAL`, `INF`, `INFEQ`, `SUP`, `SUPEQ`, `AND`, `OR`, `NEG` e `NOT`. O operador `.NE.` é implementado com `EQUAL` seguido de `NOT`. No final do programa, é emitida a instrução `STOP`.

6 Testes

A validação foi realizada através de testes unitários definidos no ficheiro `test_compiler.py`, recorrendo à biblioteca standard `unittest` do Python. Este ficheiro inclui funções auxiliares para executar a análise léxica, sintática e semântica, o que permite testar as diferentes fases do compilador de forma isolada.

Foram definidos 32 testes, distribuídos por cinco grupos: lexer, parser, análise semântica, geração de código e fluxo global do compilador. Os testes léxicos verificam o reconhecimento de palavras reservadas, identificadores, strings e operadores lógicos e relacionais. Os testes sintáticos validam a estrutura geral da AST, as declarações de arrays, as listas de entrada e saída, a instrução `WRITE`, as referências a arrays e a deteção de erros sintáticos.

Os testes semânticos verificam programas corretos e vários casos de erro, como variáveis não declaradas, incompatibilidades de tipos, labels inexistentes, labels de `DO` que não correspondem a `CONTINUE`, uso incorreto de operadores lógicos e erros associados a arrays. Os testes de geração de código verificam a presença de instruções esperadas da VM para atribuições, expressões, leitura, escrita, condicionais, ciclos, strings e arrays. Estes testes não executam a VM, apenas validam o código gerado.

Para além dos testes automáticos, o projeto inclui exemplos completos na pasta `examples`, acompanhados dos respetivos ficheiros `.vm` gerados pelo compilador. Estes exemplos foram escolhidos para cobrir diferentes partes do subconjunto implementado, sem recorrer a funcionalidades que não são suportadas pelo código.

Entre os exemplos principais encontram-se `hello.f`, que demonstra o caso mínimo de saída textual, `factorial.f`, que inclui declarações, atribuições, ciclo `DO`, `CONTINUE` e `PRINT`, `logic_if.f`, que demonstra operadores relacionais e lógicos e estruturas condicionais, `read_else.f`, que combina leitura, atribuição lógica e `IF-THEN-ELSE`, e `array_demo.f`, que demonstra arrays com acessos indexados, ciclos e saída mista.

Para tornar a validação mais explícita, apresenta-se em seguida o código VM gerado para dois dos exemplos principais:

hello.f

```
PUSHS "Ola"
WRITES
STOP
```

logic_if.f

```
PUSHN 2
PUSHI 3
STOREG 0
```



```
PUSHI 1
STOREG 1
PUSHG 0
PUSHI 0
SUP
PUSHG 1
AND
JZ IF_END_2
PUSHS "Condition true"
WRITES
IF_END_2:
STOP
```

A existência destes exemplos, juntamente com os ficheiros `.vm` correspondentes, contribui para cumprir o requisito relativo à disponibilização de programas de exemplo e do código gerado pela máquina virtual.

Para executar os testes, deve ser usado o seguinte comando, a partir da pasta do projeto:

```
python -m unittest -v
```

7 Instruções de Utilização do Compilador

O compilador é executado a partir do ficheiro `fcompiler.py`, recebendo como argumento o caminho para o ficheiro fonte:

```
python fcompiler.py examples\<ficheiro.f>
```

Se o número de argumentos for inválido, é apresentada a mensagem de uso definida no código. Se o ficheiro indicado não existir, é apresentada uma mensagem de erro. Quando a compilação é bem-sucedida, o código VM é escrito no `stdout` e a mensagem de sucesso

é escrita no `stderr`. Se ocorrerem erros léxicos, sintáticos, semânticos ou internos, a compilação falha.

8 Conclusão

O compilador desenvolvido implementa, de forma modular, as principais fases de um compilador para um subconjunto de Fortran 77. A divisão entre análise léxica, análise sintática, análise semântica e geração de código contribui para uma organização mais clara da solução, permitindo separar as responsabilidades de cada módulo.

A implementação suporta variáveis escalares e arrays unidimensionais, expressões aritméticas, relacionais e lógicas, estruturas condicionais, ciclos `DO` com incremento unitário, labels, saltos e operações básicas de entrada e saída. A geração de código é feita diretamente para a máquina virtual, sem uma etapa autónoma de otimização.

O código não implementa funcionalidades mais avançadas de Fortran 77, como funções, sub-rotinas, chamadas de funções em expressões ou formato de colunas fixas. Ainda assim, o conjunto desenvolvido permite demonstrar a base do projeto da unidade curricular de Processamento de Linguagens. Os testes unitários validam as principais funcionalidades implementadas e confirmam o comportamento esperado nos casos previstos.